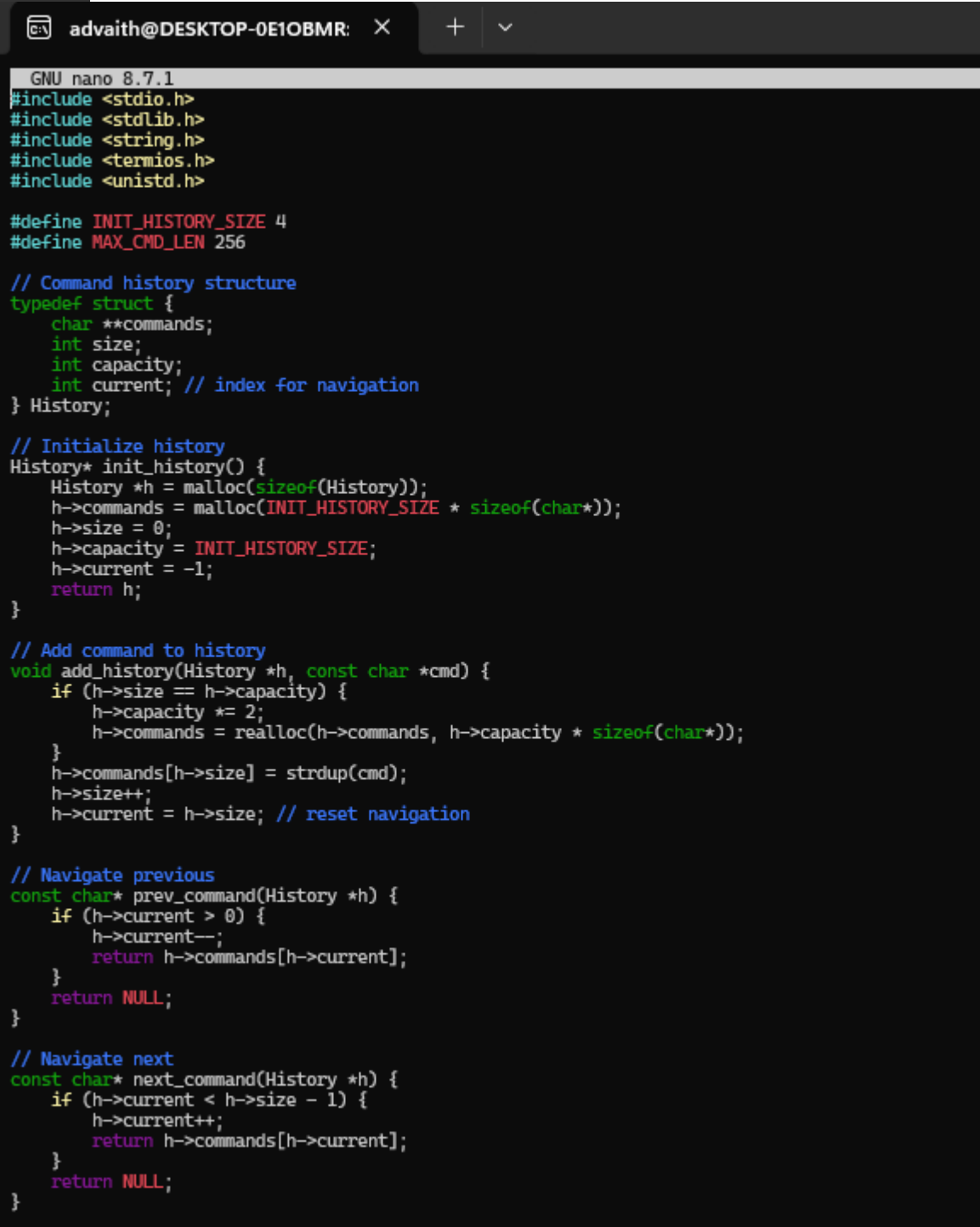


(SKILL Content Delivery Concepts)

1. To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

CODE:-



```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

#define INIT_HISTORY_SIZE 4
#define MAX_CMD_LEN 256

// Command history structure
typedef struct {
    char **commands;
    int size;
    int capacity;
    int current; // index for navigation
} History;

// Initialize history
History* init_history() {
    History *h = malloc(sizeof(History));
    h->commands = malloc(INIT_HISTORY_SIZE * sizeof(char*));
    h->size = 0;
    h->capacity = INIT_HISTORY_SIZE;
    h->current = -1;
    return h;
}

// Add command to history
void add_history(History *h, const char *cmd) {
    if (h->size == h->capacity) {
        h->capacity *= 2;
        h->commands = realloc(h->commands, h->capacity * sizeof(char*));
    }
    h->commands[h->size] = strdup(cmd);
    h->size++;
    h->current = h->size; // reset navigation
}

// Navigate previous
const char* prev_command(History *h) {
    if (h->current > 0) {
        h->current--;
        return h->commands[h->current];
    }
    return NULL;
}

// Navigate next
const char* next_command(History *h) {
    if (h->current < h->size - 1) {
        h->current++;
        return h->commands[h->current];
    }
    return NULL;
}
```

```
advaith@DESKTOP-0E1OBMR: X + v
GNU nano 8.7.1 TestRecallfun
raw = *orig;
raw.c_lflag &= ~(ICANON | ECHO);
tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}

// Restore terminal
void disable_raw_mode(struct termios *orig) {
    tcsetattr(STDIN_FILENO, TCSAFLUSH, orig);
}

int main() {
    History *history = init_history();
    struct termios orig;
    enable_raw_mode(&orig);

    char buffer[MAX_CMD_LEN];
    int pos = 0;

    printf("Simple shell (use ↑ ↓ for history, Enter to submit, Ctrl+C to quit)\n");

    while (1) {
        int c = getchar();
        if (c == '\n') {
            buffer[pos] = '\0';
            if (pos > 0) {
                add_history(history, buffer);
                printf("\n[Stored]: %s\n", buffer);
                pos = 0;
            }
            printf("> ");
        } else if (c == 27) { // Escape sequence
            getchar(); // skip '['
            int arrow = getchar();
            const char *cmd = NULL;
            if (arrow == 'A') cmd = prev_command(history); // Up
            else if (arrow == 'B') cmd = next_command(history); // Down
            if (cmd) {
                printf("\r> %s", cmd);
                fflush(stdout);
                strncpy(buffer, cmd, MAX_CMD_LEN);
                pos = strlen(cmd);
            }
        } else {
            if (pos < MAX_CMD_LEN - 1) {
                buffer[pos++] = c;
                putchar(c);
            }
        }
    }

    disable_raw_mode(&orig);
    free_history(history);
    return 0;
}
```

OUTPUT:-

```
advaith@DESKTOP-0E1OBMR:~/SKILL$ gcc TestRecallFunctionality.c -o program
advaith@DESKTOP-0E1OBMR:~/SKILL$ ./program
Simple shell (use ↑ ↓ for history, Enter to submit, Ctrl+C to quit)
> hello
[Stored]: > hello
> $ valgrind ./a.out
```

2. To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

CODE:-

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

// Token types
typedef enum {
    TOKEN_WORD,
    TOKEN_PIPE,
    TOKEN_END,
} TokenType;

// Token structure
typedef struct Token {
    TokenType type;
    char *value;
    struct Token *next;
} Token;

// Parse tree node
typedef struct Node {
    char *command;
    struct Node *pipe_to;
} Node;

// Create a new token
Token* new_token(TokenType type, const char *val) {
    Token *t = malloc(sizeof(Token));
    t->type = type;
    t->value = val ? strdup(val) : NULL;
    t->next = NULL;
    return t;
}

// Free tokens
void free_tokens(Token *t) {
    while (t) {
        Token *next = t->next;
        free(t->value);
        free(t);
        t = next;
    }
}

// Tokenizer: split input by whitespace and delimiters
Token* tokenize(const char *input) {
    Token *head = NULL, *tail = NULL;
    const char *p = input;
    while (*p) {
        while (isspace(*p)) p++; // skip whitespace
        if (*p == '\0') break;

        if (*p == '|') {
            Token *t = new_token(TOKEN_PIPE, "|");
            if (!head) head = t; else tail->next = t;
        }
    }
}
```

```
GNU nano 8.7.1
node->command = strdup(t->value);
node->pipe_to = NULL;
if (!root) root = node;
if (current) current->pipe_to = node;
current = node;
} else if (t->type == TOKEN_PIPE) {
    // Expect next word
    if (!t->next || t->next->type != TOKEN_WORD) {
        fprintf(stderr, "Syntax error: pipe without command\n");
        return NULL;
    }
}
t = t->next;
}
return root;
}

// Free parse tree
void free_tree(Node *n) {
    while (n) {
        Node *next = n->pipe_to;
        free(n->command);
        free(n);
        n = next;
    }
}

// Debug: print parse tree
void print_tree(Node *n) {
    while (n) {
        printf("Command: %s\n", n->command);
        if (n->pipe_to) printf(" |-> ");
        n = n->pipe_to;
    }
}

int main() {
    char input[256];
    printf("Enter command line: ");
    if (!fgets(input, sizeof(input), stdin)) return 1;
    input[strcspn(input, "\n")] = '\0'; // remove newline

    Token *tokens = tokenize(input);
    Node *tree = parse(tokens);

    if (tree) {
        printf("Parse tree:\n");
        print_tree(tree);
        free_tree(tree);
    }

    free_tokens(tokens);
    return 0;
}
```

OUTPUT:

```
advaith@DESKTOP-0E1OBMR:~/SKILL$ nano DebugParsing.c
advaith@DESKTOP-0E1OBMR:~/SKILL$ gcc DebugParsing.c -o program
advaith@DESKTOP-0E1OBMR:~/SKILL$ ./program
Enter command line: ls -l | grep txt | sort
Parse tree:
Command: ls
|-> Command: -l
|-> Command: grep
|-> Command: txt
|-> Command: sort
```

3. To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

CODE:-

```
GNU nano 8.7.1 TestEdo
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

// Token types
typedef enum {
    TOKEN_WORD,
    TOKEN_QUOTED_SINGLE,
    TOKEN_QUOTED_DOUBLE,
    TOKEN_END
} TokenType;

// Token structure
typedef struct Token {
    TokenType type;
    char *value;
    struct Token *next;
} Token;

// Create new token
Token* new_token(TokenType type, const char *val) {
    Token *t = malloc(sizeof(Token));
    t->type = type;
    t->value = val ? strdup(val) : NULL;
    t->next = NULL;
    return t;
}

// Free tokens
void free_tokens(Token *t) {
    while (t) {
        Token *next = t->next;
        free(t->value);
        free(t);
        t = next;
    }
}

// Tokenizer with single/double quote handling
Token* tokenize(const char *input) {
    Token *head = NULL, *tail = NULL;
    const char *p = input;

    while (*p) {
        while (isspace(*p)) p++; // skip whitespace
        if (*p == '\0') break;

        if (*p == '\'') { // single quotes
            p++;
            const char *start = p;
            while (*p && *p != '\0') p++;
            int len = p - start;
            char buf[256];
            strncpy(buf, start, len);
        }
    }
}
```

```
advaith@DESKTOP-0E1OBMR: X + v
GNU nano 8.7.1
const char *start = p;
while (*p && *p != '"') p++;
int len = p - start;
char buf[256];
strncpy(buf, start, len);
buf[len] = '\0';
Token *t = new_token(TOKEN_QUOTED_DOUBLE, buf);
if (!head) head = t; else tail->next = t;
tail = t;
if (*p == '"') p++; // skip closing
} else { // normal word
const char *start = p;
while (*p && !isspace(*p) && *p != '\\' && *p != '"') p++;
int len = p - start;
char buf[256];
strncpy(buf, start, len);
buf[len] = '\0';
Token *t = new_token(TOKEN_WORD, buf);
if (!head) head = t; else tail->next = t;
tail = t;
}
}

Token *end = new_token(TOKEN_END, NULL);
if (!head) head = end; else tail->next = end;
return head;
}

// Debug print tokens
void print_tokens(Token *t) {
while (t && t->type != TOKEN_END) {
switch (t->type) {
case TOKEN_WORD: printf("WORD: %s\n", t->value); break;
case TOKEN_QUOTED_SINGLE: printf("SINGLE QUOTED: '%s'\n", t->value); break;
case TOKEN_QUOTED_DOUBLE: printf("DOUBLE QUOTED: \"%s\"\n", t->value); break;
default: break;
}
t = t->next;
}
}

int main() {
char input[256];
printf("Enter command: ");
if (!fgets(input, sizeof(input), stdin)) return 1;
input[strcspn(input, "\n")] = '\0'; // remove newline

Token *tokens = tokenize(input);
printf("Parsed tokens:\n");
print_tokens(tokens);

free_tokens(tokens);
return 0;
}
```

OUTPUT:-

```
advaith@DESKTOP-0E1OBMR:~/SKILL$ nano TestEdgeCases.c
advaith@DESKTOP-0E1OBMR:~/SKILL$ gcc TestEdgeCases.c -o program
advaith@DESKTOP-0E1OBMR:~/SKILL$ ./program
Enter command: echo 'Hello $USER'
Parsed tokens:
WORD: echo
SINGLE QUOTED: 'Hello $USER'
```

4. To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

CODE:-

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

// Split input into arguments, handling escape sequences
char** tokenize(const char *input, int *argc) {
    int capacity = 8;
    char **args = malloc(capacity * sizeof(char*));
    *argc = 0;

    const char *p = input;
    char buf[256];
    int pos = 0;

    while (*p) {
        if (*p == ' ') {
            if (pos > 0) {
                buf[pos] = '\0';
                args[(*argc)++] = strdup(buf);
                pos = 0;
                if (*argc >= capacity) {
                    capacity *= 2;
                    args = realloc(args, capacity * sizeof(char*));
                }
            }
            p++;
        } else if (*p == '\\') { // escape sequence
            p++;
            if (*p) {
                buf[pos++] = *p; // preserve escaped char
                p++;
            }
        } else {
            buf[pos++] = *p++;
        }
    }

    if (pos > 0) {
        buf[pos] = '\0';
        args[(*argc)++] = strdup(buf);
    }

    args[*argc] = NULL; // execvp requires NULL-terminated array
    return args;
}

// Free arguments
void free_args(char **args, int argc) {
    for (int i = 0; i < argc; i++) {
        free(args[i]);
    }
    free(args);
}
```

```

int argc;
char **args = tokenize(input, &argc);

if (argc == 0) {
    fprintf(stderr, "Empty command\n");
    free_args(args, argc);
    return 1;
}

pid_t pid = fork();
if (pid == 0) {
    // Child process
    execvp(args[0], args);
    perror("Execution failed");
    exit(1);
} else if (pid > 0) {
    // Parent process
    int status;
    waitpid(pid, &status, 0);
    if (WIFEXITED(status)) {
        printf("Child exited with status %d\n", WEXITSTATUS(status));
    } else {
        printf("Child terminated abnormally\n");
    }
} else {
    perror("Fork failed");
}

free_args(args, argc);
return 0;
}

```

OUTPUT:

```

advaith@DESKTOP-0E10BMR:~/SKILL$ nano TestComplexInputs.c
advaith@DESKTOP-0E10BMR:~/SKILL$ gcc TestComplexInputs.c -o program
advaith@DESKTOP-0E10BMR:~/SKILL$ ./program
Enter command: echo hello\ world
hello world
Child exited with status 0
advaith@DESKTOP-0E10BMR:~/SKILL$ |

```

5. Process Synchronization and Child Process Monitoring Using waitpid().

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

OUTPUT:

```

advaith@DESKTOP-0E10BMR:~/SKILL$ gcc TestCommandResolution.c -o program
advaith@DESKTOP-0E10BMR:~/SKILL$ ./program
Enter command to run: ls
DebugParsing.c  TestCommandResolution.c  TestEdgeCases.c          commands  program
Test           TestComplexInputs.c       TestRecallFunctionality.c commands.c
Child exited with status 0

```

6. To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic.

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

CODE:-

```
GNU nano 8.7.1 TestExpansio
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <ctype.h>
#include <sys/wait.h>

// Expand variables like $HOME or $USER
char* expand_variables(const char *input) {
    char buf[512];
    int pos = 0;
    for (int i = 0; input[i]; i++) {
        if (input[i] == '$') {
            i++;
            char var[128];
            int vpos = 0;
            while (input[i] && (isalnum(input[i]) || input[i] == '_')) {
                var[vpos++] = input[i++];
            }
            var[vpos] = '\0';
            i--; // step back one
            char *val = getenv(var);
            if (val) {
                int len = strlen(val);
                strncpy(&buf[pos], val, len);
                pos += len;
            } else {
                // Undefined variable -> empty string
            }
        } else {
            buf[pos++] = input[i];
        }
    }
    buf[pos] = '\0';
    return strdup(buf);
}

// Built-in command handler
int builtin_cd(char **args) {
    if (!args[1]) {
        fprintf(stderr, "cd: missing argument\n");
        return 1;
    }
    if (chdir(args[1]) != 0) {
        perror("cd failed");
        return 1;
    }
    return 0;
}

int builtin_exit(char **args) {
    exit(0);
}
```

```
advaith@DESKTOP-0E1OBMR: X + v
GNU nano 8.7.1 TestE
    free(args);
}

int main() {
    char input[256];
    while (1) {
        printf("> ");
        if (!fgets(input, sizeof(input), stdin)) break;
        input[strcspn(input, "\n")] = '\0';

        // Expand variables
        char *expanded = expand_variables(input);

        // Tokenize
        char **args = tokenize(expanded);
        free(expanded);

        if (!args[0]) {
            free_args(args);
            continue;
        }

        // Check built-ins
        int handled = 0;
        for (int i = 0; builtins[i].name; i++) {
            if (strcmp(args[0], builtins[i].name) == 0) {
                builtins[i].func(args);
                handled = 1;
                break;
            }
        }

        if (!handled) {
            // External command
            pid_t pid = fork();
            if (pid == 0) {
                execvp(args[0], args);
                perror("exec failed");
                exit(1);
            } else if (pid > 0) {
                int status;
                waitpid(pid, &status, 0);
                if (WIFEXITED(status)) {
                    printf("Process exited with %d\n", WEXITSTATUS(status));
                }
            } else {
                perror("fork failed");
            }
        }

        free_args(args);
    }
    return 0;
}
```

OUTPUT:

```
advaith@DESKTOP-0E1OBMR:~/SKILL$ nano TestExpansionLogic.c
advaith@DESKTOP-0E1OBMR:~/SKILL$ gcc TestExpansionLogic.c -o program
advaith@DESKTOP-0E1OBMR:~/SKILL$ ./program
> echo $HOME
/home/advaith
> cd /tmp
```

7. To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

CODE:-

```
GNU nano 8.7.1
advaith@DESKTOP-0E1OBMR: X + v

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

// Built-in command prototypes
int builtin_cd(char **args);
int builtin_pwd(char **args);
int builtin_exit(char **args);

// Dispatch table
struct builtin {
    const char *name;
    int (*func)(char **args);
};

struct builtin builtins[] = {
    {"cd", builtin_cd},
    {"pwd", builtin_pwd},
    {"exit", builtin_exit},
    {NULL, NULL}
};

// Track previous directory
char prev_dir[256] = "";

// Built-in: change directory
int builtin_cd(char **args) {
    char cwd[256];
    getcwd(cwd, sizeof(cwd));

    if (!args[1]) {
        fprintf(stderr, "cd: missing argument\n");
        return 1;
    }

    if (strcmp(args[1], "-") == 0) {
        if (strlen(prev_dir) == 0) {
            fprintf(stderr, "cd: no previous directory\n");
            return 1;
        }
        if (chdir(prev_dir) != 0) {
            perror("cd failed");
            return 1;
        }
        printf("%s\n", prev_dir);
    } else {
        if (chdir(args[1]) != 0) {
            perror("cd failed");
            return 1;
        }
    }

    strcpy(prev_dir, cwd);
}
```

```
GNU nano 8.7.1 TestNavigation.c
args[argc] = NULL;
free(copy);
return args;
}

void free_args(char **args) {
    for (int i = 0; args[i]; i++) free(args[i]);
    free(args);
}

int main() {
    char input[256];
    while (1) {
        printf("> ");
        if (!fgets(input, sizeof(input), stdin)) break;
        input[strcspn(input, "\n")] = '\0';

        char **args = tokenize(input);
        if (!args[0]) {
            free_args(args);
            continue;
        }

        // Check built-ins
        int handled = 0;
        for (int i = 0; builtins[i].name; i++) {
            if (strcmp(args[0], builtins[i].name) == 0) {
                builtins[i].func(args);
                handled = 1;
                break;
            }
        }

        // External command
        if (!handled) {
            pid_t pid = fork();
            if (pid == 0) {
                execvp(args[0], args);
                perror("exec failed");
                exit(1);
            } else if (pid > 0) {
                int status;
                waitpid(pid, &status, 0);
                if (WIFEXITED(status))
                    printf("Process exited with %d\n", WEXITSTATUS(status));
            } else {
                perror("fork failed");
            }
        }

        free_args(args);
    }
    return 0;
}
```

OUTPUT:

```
advaith@DESKTOP-0E1OBMR:~/SKILL$ nano TestNavigation.c
advaith@DESKTOP-0E1OBMR:~/SKILL$ gcc TestNavigation.c -o program
advaith@DESKTOP-0E1OBMR:~/SKILL$ ./program
> pwd
/home/advaith/SKILL
> cd -
cd: no previous directory
> badcmd
exec failed: No such file or directory
Process exited with 1
> cd /tmp
```

8. To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

CODE:

```
advaith@DESKTOP-0E10BMR: X + v
GNU nano 2.7.1 Retrieve.c

int main() {
    char input[256];
    while (1) {
        printf("> ");
        if (!fgets(input, sizeof(input), stdin)) break;
        input[strcspn(input, "\n")] = '\0';

        char **args = tokenize(input);
        if (!args[0]) {
            free_args(args);
            continue;
        }

        // Check built-ins
        int handled = 0;
        for (int i = 0; builtins[i].name; i++) {
            if (strcmp(args[0], builtins[i].name) == 0) {
                builtins[i].func(args);
                handled = 1;
                break;
            }
        }

        // External command
        if (!handled) {
            pid_t pid = fork();
            if (pid == 0) {
                execvp(args[0], args);
                perror("exec failed");
                exit(1);
            } else if (pid > 0) {
                int status;
                waitpid(pid, &status, 0);
                if (WIFEXITED(status))
                    printf("Process exited with %d\n", WEXITSTATUS(status));
            } else {
                perror("fork failed");
            }
        }

        free_args(args);
    }

    return 0;
}
```

OUTPUT:

```
advaith@DESKTOP-0E10BMR:~/SKILL$ nano Retrieve.c
advaith@DESKTOP-0E10BMR:~/SKILL$ gcc Retrieve.c -o program
advaith@DESKTOP-0E10BMR:~/SKILL$ ./program
> pwd
/home/advaith/SKILL
> export MYVAR=hello
Exported MYVAR=hello
> echo $MYVAR
$MYVAR
Process exited with 0
> exit
Exiting shell...
```

9. To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define HISTORY_CAPACITY 10

// Command history
char *history[HISTORY_CAPACITY];
int history_size = 0;

// Add command to history
void add_history(const char *cmd) {
    if (history_size < HISTORY_CAPACITY) {
        history[history_size++] = strdup(cmd);
    } else {
        free(history[0]);
        for (int i = 1; i < HISTORY_CAPACITY; i++) {
            history[i-1] = history[i];
        }
        history[HISTORY_CAPACITY-1] = strdup(cmd);
    }
}

// Display history
void show_history() {
    printf("Command History:\n");
    for (int i = 0; i < history_size; i++) {
        printf("%d: %s\n", i+1, history[i]);
    }
}

// Free history
void free_history() {
    for (int i = 0; i < history_size; i++) {
        free(history[i]);
    }
}

// Execute pipeline
void execute_pipeline(char *input) {
    char *commands[10];
    int cmd_count = 0;

    char *tok = strtok(input, "|");
    while (tok && cmd_count < 10) {
        commands[cmd_count++] = strdup(tok);
        tok = strtok(NULL, "|");
    }

    int pipes[2*cmd_count];
    for (int i = 0; i < cmd_count-1; i++) {
```

CODE:

```
for (int i = 0; i < cmd_count-1; i++) {
    if (pipe(pipes + i*2) < 0) {
        perror("pipe failed");
        exit(1);
    }
}

for (int i = 0; i < cmd_count; i++) {
    pid_t pid = fork();
    if (pid == 0) {
        // Redirect input
        if (i > 0) {
            dup2(pipes[(i-1)*2], 0);
        }
        // Redirect output
        if (i < cmd_count-1) {
            dup2(pipes[i*2+1], 1);
        }
        // Close all pipes
        for (int j = 0; j < 2*(cmd_count-1); j++) {
            close(pipes[j]);
        }
        // Tokenize command
        char *args[10];
        int argc = 0;
        char *arg = strtok(commands[i], " \t");
        while (arg && argc < 9) {
            args[argc++] = arg;
            arg = strtok(NULL, " \t");
        }
        args[argc] = NULL;
        execvp(args[0], args);
        perror("exec failed");
        exit(1);
    }
}

// Close all pipes in parent
for (int j = 0; j < 2*(cmd_count-1); j++) {
    close(pipes[j]);
}

// Wait for children
for (int i = 0; i < cmd_count; i++) {
    int status;
    wait(&status);
}

for (int i = 0; i < cmd_count; i++) {
    free(commands[i]);
}
}

int main() {
    char input[256];
```

```

    int status;
    wait(&status);
}

for (int i = 0; i < cmd_count; i++) {
    free(commands[i]);
}
}

int main() {
    char input[256];
    while (1) {
        printf("> ");
        if (!fgets(input, sizeof(input), stdin)) break;
        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0) continue;

        add_history(input);

        if (strcmp(input, "history") == 0) {
            show_history();
            continue;
        }

        execute_pipeline(input);

        free_history();
        return 0;
    }
}

```

OUTPUT:

```

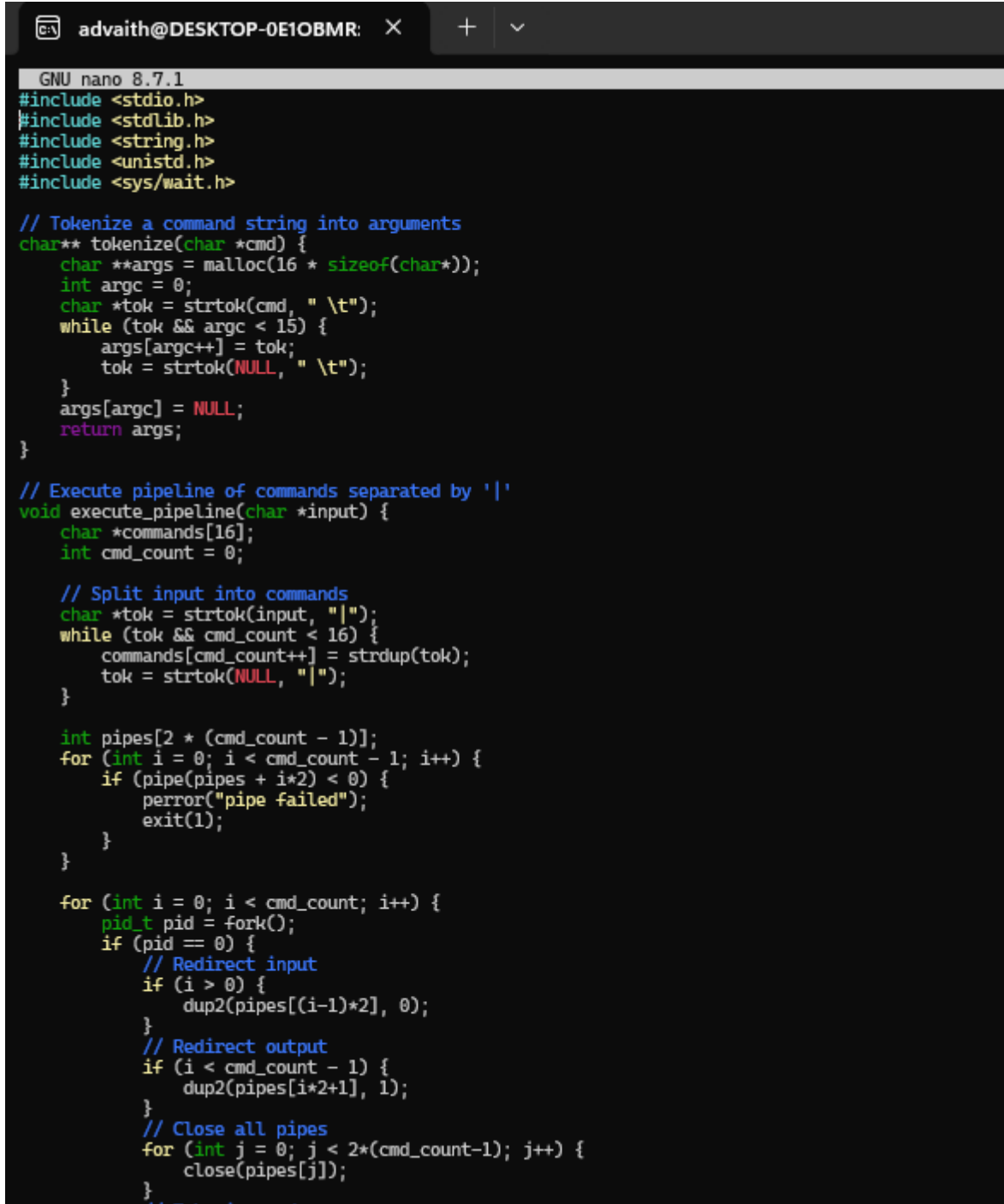
advaith@DESKTOP-0E1OBMR: X + v
advaith@DESKTOP-0E1OBMR:~$ nano pipeline.c
advaith@DESKTOP-0E1OBMR:~$ gcc pipeline.c -o program
advaith@DESKTOP-0E1OBMR:~$ ./program
> ls -l | grep .c | sort
-rw----- 1 advaith advaith    1 Aug 11 06:34 transitions.c.save
-rw-r--r-- 1 advaith advaith   77 Aug  7 08:51 hello.c
-rw-r--r-- 1 advaith advaith  104 Aug  7 09:01 fork.c
-rw-r--r-- 1 advaith advaith  110 Aug  7 08:57 ID.c
-rw-r--r-- 1 advaith advaith  110 Aug  7 08:59 processid.c
-rw-r--r-- 1 advaith advaith  112 Aug 14 09:38 PROCESSID.c
-rw-r--r-- 1 advaith advaith  152 Aug 14 10:14 concurrently.c
-rw-r--r-- 1 advaith advaith  158 Aug  7 08:53 username.c
-rw-r--r-- 1 advaith advaith  158 Aug 14 10:10 linuxcommand.c
-rw-r--r-- 1 advaith advaith  159 Aug  7 09:09 waittwo.c
-rw-r--r-- 1 advaith advaith  163 Aug 14 10:05 MULTIPLECHILD.c
-rw-r--r-- 1 advaith advaith  229 Aug 14 09:46 pidchildprocess.c
-rw-r--r-- 1 advaith advaith  229 Aug 14 09:50 childPROCESS.c
-rw-r--r-- 1 advaith advaith  229 Aug 14 09:52 CHILDPROCESS.c
-rw-r--r-- 1 advaith advaith  230 Aug  7 09:04 parentchild.c
-rw-r--r-- 1 advaith advaith  260 Aug 14 09:56 PARENTPID.c
-rw-r--r-- 1 advaith advaith  264 Aug  7 09:11 forkexec.c
-rw-r--r-- 1 advaith advaith  274 Aug  7 09:08 wait.c
-rw-r--r-- 1 advaith advaith  287 Aug  7 09:06 parentchildpid.c
-rw-r--r-- 1 advaith advaith  532 Aug 11 03:59 controltransitions.c
-rw-r--r-- 1 advaith advaith  532 Aug 17 04:12 systemcalls.c
-rw-r--r-- 1 advaith advaith  557 Aug  7 09:28 childprocess.c
-rw-r--r-- 1 advaith advaith 2895 Sep 17 13:58 pipeline.c
drwxr-xr-x 2 advaith advaith 4096 Aug  7 09:11 forkexec
drwxr-xr-x 2 advaith advaith 4096 Aug  7 09:53 controltransitions

```

10. To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

CODE:



```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

// Tokenize a command string into arguments
char** tokenize(char *cmd) {
    char **args = malloc(16 * sizeof(char*));
    int argc = 0;
    char *tok = strtok(cmd, " \\t");
    while (tok && argc < 15) {
        args[argc++] = tok;
        tok = strtok(NULL, " \\t");
    }
    args[argc] = NULL;
    return args;
}

// Execute pipeline of commands separated by '|'
void execute_pipeline(char *input) {
    char *commands[16];
    int cmd_count = 0;

    // Split input into commands
    char *tok = strtok(input, "|");
    while (tok && cmd_count < 16) {
        commands[cmd_count++] = strdup(tok);
        tok = strtok(NULL, "|");
    }

    int pipes[2 * (cmd_count - 1)];
    for (int i = 0; i < cmd_count - 1; i++) {
        if (pipe(pipes + i*2) < 0) {
            perror("pipe failed");
            exit(1);
        }
    }

    for (int i = 0; i < cmd_count; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            // Redirect input
            if (i > 0) {
                dup2(pipes[(i-1)*2], 0);
            }
            // Redirect output
            if (i < cmd_count - 1) {
                dup2(pipes[i*2+1], 1);
            }
            // Close all pipes
            for (int j = 0; j < 2*(cmd_count-1); j++) {
                close(pipes[j]);
            }
            // Tokenize and exec
        }
    }
}
```

```

        perror("exec failed");
        exit(1);
    }
}

// Close all pipes in parent
for (int j = 0; j < 2*(cmd_count-1); j++) {
    close(pipes[j]);
}

// Wait for all children
for (int i = 0; i < cmd_count; i++) {
    int status;
    wait(&status);
}

for (int i = 0; i < cmd_count; i++) {
    free(commands[i]);
}
}

int main() {
    char input[256];
    while (1) {
        printf("> ");
        if (!fgets(input, sizeof(input), stdin)) break;
        input[strcspn(input, "\n")] = '\0';
        if (strlen(input) == 0) continue;

        execute_pipeline(input);
    }
    return 0;
}

```

OUTPUT:

```

advaith@DESKTOP-0E10BMR:~$ nano DataFlow.c
advaith@DESKTOP-0E10BMR:~$ gcc DataFlow.c -o program
advaith@DESKTOP-0E10BMR:~$ ./program
> ls -l | grep .c | sort | wc -l
26
> echo hello world | tr a-z A-Z
HELLO WORLD
> |

```

-
11. To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection.

```
advaith@DESKTOP-0E1OBMR: X + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

// Tokenize command into arguments
char** tokenize(char *cmd) {
    char **args = malloc(16 * sizeof(char*));
    int argc = 0;
    char *tok = strtok(cmd, " \\t");
    while (tok && argc < 15) {
        args[argc++] = tok;
        tok = strtok(NULL, " \\t");
    }
    args[argc] = NULL;
    return args;
}

void execute_command(char *input) {
    char *infile = NULL, *outfile = NULL;
    int append = 0;

    // Parse redirection
    char *cmd = strdup(input);
    char *tok = strtok(cmd, " ");
    char command[256] = "";
    while (tok) {
        if (strcmp(tok, "<") == 0) {
            tok = strtok(NULL, " ");
            infile = tok;
        } else if (strcmp(tok, ">") == 0) {
            tok = strtok(NULL, " ");
            outfile = tok;
            append = 0;
        } else if (strcmp(tok, ">>") == 0) {
            tok = strtok(NULL, " ");
            outfile = tok;
            append = 1;
        } else {
            strcat(command, tok);
            strcat(command, " ");
        }
        tok = strtok(NULL, " ");
    }
}
```

CODE:

```
advaith@DESKTOP-0E1OBMR: X + v
GNU nano 8.7.1
}

// Output redirection
if (outfile) {
    int fd;
    if (append)
        fd = open(outfile, O_WRONLY | O_CREAT | O_APPEND, 0644);
    else
        fd = open(outfile, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0) {
        perror("Cannot open output file");
        exit(1);
    }
    dup2(fd, STDOUT_FILENO);
    close(fd);
}

execvp(args[0], args);
perror("exec failed");
exit(1);
} else if (pid > 0) {
    int status;
    waitpid(pid, &status, 0);
    if (WIFEXITED(status))
        printf("Process exited with %d\n", WEXITSTATUS(status));
} else {
    perror("fork failed");
}

free(args);
free(cmd);
}

int main() {
    char input[256];
    while (1) {
        printf("> ");
        if (!fgets(input, sizeof(input), stdin)) break;
        input[strcspn(input, "\n")] = '\0';
        if (strlen(input) == 0) continue;

        execute_command(input);
    }
    return 0;
}
```

```
advaith@DESKTOP-0E1OBMR:~$ nano FileInputs.c
advaith@DESKTOP-0E1OBMR:~$ gcc FileInputs.c -o program
advaith@DESKTOP-0E1OBMR:~$ ./program
> cat < input.txt
Cannot open input file: No such file or directory
Process exited with 1
> echo hello > out.txt
Process exited with 0
> |
```

OUTPUT:

-
12. To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

// Tokenize command into arguments
char** tokenize(char *cmd) {
    char **args = malloc(16 * sizeof(char*));
    int argc = 0;
    char *tok = strtok(cmd, " \\t");
    while (tok && argc < 15) {
        args[argc++] = tok;
        tok = strtok(NULL, " \\t");
    }
    args[argc] = NULL;
    return args;
}

void execute_command(char *input) {
    char *outfile = NULL, *errfile = NULL;
    int append_out = 0, append_err = 0;

    // Parse redirection
    char *cmd = strdup(input);
    char *tok = strtok(cmd, " ");
    char command[256] = "";
    while (tok) {
        if (strcmp(tok, ">") == 0) {
            tok = strtok(NULL, " ");
            outfile = tok;
            append_out = 0;
        } else if (strcmp(tok, ">>") == 0) {
            tok = strtok(NULL, " ");
            outfile = tok;
            append_out = 1;
        } else if (strcmp(tok, "2>") == 0) {
            tok = strtok(NULL, " ");
            errfile = tok;
            append_err = 0;
        } else if (strcmp(tok, "2>>") == 0) {
            tok = strtok(NULL, " ");
            errfile = tok;
            append_err = 1;
        } else {
            strcat(command, tok);
        }
    }
}
```

```

GNU nano 8.7.1
}

// Error redirection
if (errfile) {
    int fd;
    if (append_err)
        fd = open(errfile, O_WRONLY | O_CREAT | O_APPEND, 0644);
    else
        fd = open(errfile, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0) {
        perror("Cannot open error file");
        exit(1);
    }
    dup2(fd, STDERR_FILENO);
    close(fd);
}

execvp(args[0], args);
perror("exec failed");
exit(1);
} else if (pid > 0) {
    int status;
    waitpid(pid, &status, 0);
    if (WIFEXITED(status))
        printf("Process exited with %d\n", WEXITSTATUS(status));
} else {
    perror("fork failed");
}

free(args);
free(cmd);
}

int main() {
    char input[256];
    while (1) {
        printf("> ");
        if (!fgets(input, sizeof(input), stdin)) break;
        input[strcspn(input, "\n")] = '\0';
        if (strlen(input) == 0) continue;

        execute_command(input);
    }
    return 0;
}

```

```

advaith@DESKTOP-0E1OBMR:~$ nano Parse.c
advaith@DESKTOP-0E1OBMR:~$ gcc Parse.c -o program
advaith@DESKTOP-0E1OBMR:~$ ./program
> echo hello >> out.txt
Process exited with 0
> ls nofile 2>> errors.txt
Process exited with 2
>

```

OUTPUT:

13. To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

// Tokenize command into arguments
char** tokenize(char *cmd) {
    char **args = malloc(32 * sizeof(char*));
    int argc = 0;
    char *tok = strtok(cmd, " \\t");
    while (tok && argc < 31) {
        args[argc++] = tok;
        tok = strtok(NULL, " \\t");
    }
    args[argc] = NULL;
    return args;
}

// Execute pipeline with redirection
void execute_pipeline(char *input) {
    char *commands[16];
    int cmd_count = 0;

    // Split input into pipeline segments
    char *tok = strtok(input, "|");
    while (tok && cmd_count < 16) {
        commands[cmd_count++] = strdup(tok);
        tok = strtok(NULL, "|");
    }

    int pipes[2 * (cmd_count - 1)];
    for (int i = 0; i < cmd_count - 1; i++) {
        if (pipe(pipes + i*2) < 0) {
            perror("pipe failed");
            exit(1);
        }
    }

    for (int i = 0; i < cmd_count; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            // Handle pipe redirection
            if (i > 0) dup2(pipes[(i-1)*2], STDIN_FILENO);
            if (i < cmd_count - 1) dup2(pipes[i*2+1], STDOUT_FILENO);

            // Close all pipes
            for (int j = 0; j < 2*(cmd_count-1); j++) close(pipes[j]);

            // Parse redirection inside command
            char *outfile = NULL, *errfile = NULL;
            int append_out = 0, append_err = 0, merge_err = 0;
            char command[256] = "";
            char *tok2 = strtok(commands[i], " ");
```

```

advait@DESKTOP-0E1OBMR: X + v
GNU nano 8.7.1 ParseComplex.c *
    if (outfile) {
        int fd = open(outfile,
            O_WRONLY | O_CREAT | (append_out ? O_APPEND : O_TRUNC),
            0644);
        if (fd < 0) { perror("Cannot open output file"); exit(1); }
        dup2(fd, STDOUT_FILENO);
        close(fd);
    }

    // Error redirection
    if (errfile) {
        int fd = open(errfile,
            O_WRONLY | O_CREAT | (append_err ? O_APPEND : O_TRUNC),
            0644);
        if (fd < 0) { perror("Cannot open error file"); exit(1); }
        dup2(fd, STDERR_FILENO);
        close(fd);
    }

    // Merge stderr into stdout
    if (merge_err) {
        dup2(STDOUT_FILENO, STDERR_FILENO);
    }

    execvp(args[0], args);
    perror("exec failed");
    exit(1);
}

// Parent closes pipes
for (int j = 0; j < 2*(cmd_count-1); j++) close(pipes[j]);

// Wait for all children
for (int i = 0; i < cmd_count; i++) {
    int status;
    wait(&status);
}

for (int i = 0; i < cmd_count; i++) free(commands[i]);

int main() {
    char input[256];
    while (1) {
        printf("> ");
        if (!fgets(input, sizeof(input), stdin)) break;
        input[strcspn(input, "\n")] = '\0';
        if (strlen(input) == 0) continue;

        execute_pipeline(input);
    }
    return 0;
}

```

OUTPUT:

```

advait@DESKTOP-0E1OBMR:~$ nano ParseComplex
advait@DESKTOP-0E1OBMR:~$ nano ParseComplex.c
advait@DESKTOP-0E1OBMR:~$ gcc ParseComplex.c -o program
advait@DESKTOP-0E1OBMR:~$ ./program
> ls -l | grep .c > out.txt 2>&1
> cat missing.txt 2> errors.txt
> echo hello >> log.txt | wc -c
0
> |

```

14. To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define MAX 10

struct Job {
    int id;
    pid_t pid;
    char command[50];
    char status[15];
};

struct Job jobs[MAX];
int count = 0;

void showJobs() {
    int status;
    for (int i = 0; i < count; i++) {
        if (strcmp(jobs[i].status, "Running") == 0) {
            if (waitpid(jobs[i].pid, &status, WNOHANG) > 0)
                strcpy(jobs[i].status, "Completed");
        }
        printf("[%d] PID:%d %s %s\n",
            jobs[i].id, jobs[i].pid,
            jobs[i].command, jobs[i].status);
    }
}

int main() {
    char command[50];

    printf("Background Job Manager\n");

    while (1) {
        printf("\n> ");
        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (strcmp(command, "jobs") == 0) {
            showJobs();
            continue;
        }

        if (command[strlen(command) - 1] == '&') {
            command[strlen(command) - 1] = '\0';

            pid_t pid = fork();

            if (pid == 0) {
```

```

    }

    if (command[strlen(command) - 1] == '&') {
        command[strlen(command) - 1] = '\\0';

        pid_t pid = fork();

        if (pid == 0) {
            execlp(command, command, NULL);
            exit(1);
        }

        jobs[count].id = count + 1;
        jobs[count].pid = pid;
        strcpy(jobs[count].command, command);
        strcpy(jobs[count].status, "Running");
        count++;

        printf("[%d] Started in background\\n", count);
    }
}

return 0;
}

```

CODE:

```

> nano Detection.c
advaith@DESKTOP-0E10BMR:~$ gcc Detection.c -o program
advaith@DESKTOP-0E10BMR:~$ ./program
Background Job Manager

> sleep 5 &
[1] Started in background

> jobs
[1] PID:1098  sleep 5  Completed

> exit
advaith@DESKTOP-0E10BMR:~$ |

```

OUTPUT:

-
15. To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

struct Job {
    int id;
    pid_t pid;
    int running;
};

struct Job jobs[5];
int count = 0;

void listJobs() {
    printf("\nActive Jobs:\n");

    for (int i = 0; i < count; i++) {
        printf("[%d] PID=%d Status=%s\n",
            jobs[i].id,
            jobs[i].pid,
            jobs[i].running ? "Running" : "Completed");
    }
}

void foreground(int id) {
    for (int i = 0; i < count; i++) {
        if (jobs[i].id == id) {
            printf("Bringing Job [%d] to foreground...\n", id);

            /* Resume process */
            kill(jobs[i].pid, SIGCONT);

            /* Wait for completion */
            waitpid(jobs[i].pid, NULL, 0);

            /* Update state */
            jobs[i].running = 0;

            printf("Job [%d] completed.\n", id);
            return;
        }
    }

    printf("Job not found.\n");
}
```

```
}  
  
int main() {  
    int choice, id;  
    pid_t pid;  
  
    printf("Job Control System\n");  
  
    while (1) {  
        printf("\n1. Start Job");  
        printf("\n2. List Jobs");  
        printf("\n3. Foreground Job");  
        printf("\n4. Exit");  
        printf("\nEnter choice: ");  
        scanf("%d", &choice);  
  
        if (choice == 1) {  
            pid = fork();  
  
            if (pid == 0) {  
                execlp("sleep", "sleep", "10", NULL);  
            }  
  
            jobs[count].id = count + 1;  
            jobs[count].pid = pid;  
            jobs[count].running = 1;  
            count++;  
  
            printf("Job [%d] started.\n", count);  
        }  
  
        else if (choice == 2) {  
            listJobs();  
        }  
  
        else if (choice == 3) {  
            printf("Enter Job ID: ");  
            scanf("%d", &id);  
            foreground(id);  
        }  
  
        else if (choice == 4) {  
            break;  
        }  
  
        else {  
            printf("Invalid choice.\n");  
        }  
    }  
}
```

```

advaith@DESKTOP-0E10BMR:~$ nano ActiveJob.c
advaith@DESKTOP-0E10BMR:~$ gcc ActiveJob.c -o program
advaith@DESKTOP-0E10BMR:~$ ./program
Job Control System

1. Start Job
2. List Jobs
3. Foreground Job
4. Exit
Enter choice: 1
Job [1] started.

1. Start Job
2. List Jobs
3. Foreground Job
4. Exit
Enter choice: 8
Invalid choice.

1. Start Job
2. List Jobs
3. Foreground Job
4. Exit
Enter choice: 2

Active Jobs:
[1] PID=1168 Status=Running

```

OUTPUT:

-
16. To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery.

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

struct Job {
    int id;
    pid_t pid;
    int status;    // 1 = Running, 0 = Stopped
};

struct Job job;

void checkStatus() {
    int status;
    pid_t r = waitpid(job.pid, &status, WNOHANG | WUNTRACED);

    if (r > 0) {
        if (WIFSTOPPED(status)) {
            job.status = 0;
            printf("Job [%d] Stopped\n", job.id);
        }
        else if (WIFCONTINUED(status)) {
            job.status = 1;
            printf("Job [%d] Running\n", job.id);
        }
    }
}

int main() {
    pid_t pid;

    printf("Signal and Job Recovery System\n");

    pid = fork();

    if (pid == 0) {
        /* Create a separate process group */
        setpgid(0, 0);

        while (1) {
            printf("Child process running...\n");
            sleep(2);
        }
    }

    job.id = 1;
```

```

    }

    job.id = 1;
    job.pid = pid;
    job.status = 1;

    sleep(2);

    /* Stop the job */
    printf("\nStopping Job [1]...\n");
    kill(pid, SIGSTOP);
    sleep(1);

    checkStatus();

    /* Resume the stopped job */
    printf("\nSending SIGCONT...\n");
    kill(pid, SIGCONT);

    sleep(1);

    /* Check updated status */
    checkStatus();

    printf("\nJob resumed successfully.\n");

    /* Stop and terminate child */
    kill(pid, SIGTERM);
    waitpid(pid, NULL, 0);

    return 0;
}

```

```

advaith@DESKTOP-0E10BMR:~$ nano sigcount.c
advaith@DESKTOP-0E10BMR:~$ gcc sigcount.c -o program
advaith@DESKTOP-0E10BMR:~$ ./program
Signal and Job Recovery System
Child process running...

Child process running...
Stopping Job [1]...
Job [1] Stopped

Sending SIGCONT...
Child process running...

Job resumed successfully.
advaith@DESKTOP-0E10BMR:~$ |

```

OUTPUT:

-
17. To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

```
advaith@DESKTOP-0E1OBMR: X + v
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t child = -1;
int job_running = 0;

/* SIGINT handler */
void handle_sigint(int sig)
{
    if (child > 0 && job_running) {
        printf("\nSIGINT received. Terminating foreground job...\n");

        /* Forward SIGINT to child */
        kill(child, SIGINT);
    }
    else {
        printf("\nShell protected from SIGINT.\n");
    }
}

int main()
{
    signal(SIGINT, handle_sigint);

    printf("Simple Signal Handling Shell\n");

    while (1) {
        printf("\nEnter 1 to start job, 2 to exit: ");

        int choice;
        scanf("%d", &choice);

        if (choice == 2)
            break;

        if (choice == 1) {
            child = fork();

            if (child == 0) {
                printf("Foreground job running...\n");

                /* Simulate long-running job */
                while (1)
                    sleep(1);
            }
        }
    }
}
```

```

        /* Simulate long-running job */
        while (1)
            sleep(1);
    }

    job_running = 1;

    /* Shell waits while protecting itself from SIGINT */
    waitpid(child, NULL, 0);

    job_running = 0;
    child = -1;

    printf("Foreground job terminated.\n");
}

printf("Shell exiting...\n");
return 0;
}

```

CODE:

```

advaith@DESKTOP-0E10BMR:~$ nano sigint
advaith@DESKTOP-0E10BMR:~$ nano sigint.c
advaith@DESKTOP-0E10BMR:~$ gcc sigint.c -o program
advaith@DESKTOP-0E10BMR:~$ ./program
Simple Signal Handling Shell

Enter 1 to start job, 2 to exit: 1
Foreground job running...
2

```

OUTPUT:

-
18. To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t child;
int job_status = 1; // 1 = Running, 0 = Stopped

void handle_tstp(int sig)
{
    printf("\nSIGTSTP received. Suspending job...\n");
    kill(child, SIGSTOP);
    job_status = 0;
}

int main()
{
    int status;

    signal(SIGTSTP, handle_tstp);

    child = fork();

    if (child == 0) {
        /* Create new process group */
        setpgid(0, 0);

        while (1) {
            printf("Child process running...\n");
            sleep(2);
        }
    }

    /* Parent assigns child to its own group */
    setpgid(child, child);

    printf("Job started. PID = %d\n", child);
    printf("Press Ctrl+Z to suspend the job.\n");

    while (1) {

        waitpid(child, &status, WUNTRACED);

        if (WIFSTOPPED(status)) {
            job_status = 0;
            printf("Job stopped. Status saved.\n");
        }
    }
}
```


To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

```
GNU nano 8.7.1 con
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void logError(const char *msg)
{
    FILE *fp = fopen("error.log", "a");

    if (fp != NULL) {
        fprintf(fp, "ERROR: %s\n", msg);
        fclose(fp);
    }
}

int executeCommand(char *cmd)
{
    if (strcmp(cmd, "start") == 0) {
        printf("Command executed successfully.\n");
        return 0;
    }

    if (strcmp(cmd, "stop") == 0) {
        printf("Process stopped successfully.\n");
        return 0;
    }

    if (strcmp(cmd, "status") == 0) {
        printf("System is running.\n");
        return 0;
    }

    printf("Error: Invalid command syntax.\n");
    logError("Invalid command entered");
    return -1;
}

int main()
{
    char command[50];

    printf("Simple Error Handling System\n");

    while (1) {
        printf("\nEnter command (start/stop/status/exit): ");
        fgets(command, sizeof(command), stdin);

        command[strcspn(command, "\n")] = '\0';
    }
}
```

```

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (strlen(command) == 0) {
            printf("Error: Empty command.\n");
            logError("Empty command");
            continue;
        }

        if (executeCommand(command) != 0) {
            printf("Recovering... Please enter a valid command.\n");
        }
    }

    printf("Program terminated safely.\n");
    return 0;
}
CODE:

```

```

advaith@DESKTOP-0E10BMR:~$ nano controlerrors.c
advaith@DESKTOP-0E10BMR:~$ gcc controlerrors.c -o program
advaith@DESKTOP-0E10BMR:~$ ./program
Simple Error Handling System

Enter command (start/stop/status/exit): start
Command executed successfully.

Enter command (start/stop/status/exit): stop
Process stopped successfully.

Enter command (start/stop/status/exit): status
System is running.

Enter command (start/stop/status/exit): exit
Program terminated safely.
OUTPUT:

```

20. To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

CODE:

```
advaith@DESKTOP-0E1OBMR: X + v
GNU nano 8.7.1 dp.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, sum = 0;

    printf("Enter number of elements: ");

    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("ERROR: Invalid input\n");
        return 1;
    }

    int *arr = malloc(n * sizeof(int));

    if (arr == NULL) {
        printf("ERROR: Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        printf("Enter element %d: ", i + 1);

        if (scanf("%d", &arr[i]) != 1) {
            printf("ERROR: Invalid value\n");
            free(arr);
            return 1;
        }

        sum += arr[i];
    }

    printf("\n----- TEST REPORT ----- \n");
    printf("Elements : %d\n", n);
    printf("Sum      : %d\n", sum);
    printf("Expected : Memory released\n");

    /* Release allocated memory */
    free(arr);
    arr = NULL;

    printf("Result   : PASS\n");
    printf("Cleanup  : Successful\n");

    return 0;
}
```

```
advaith@DESKTOP-0E1OBMR:~$ nano dp.c
advaith@DESKTOP-0E1OBMR:~$ gcc dp.c -o program
advaith@DESKTOP-0E1OBMR:~$ ./program
Enter number of elements: 8
Enter element 1: 2
Enter element 2: 4
Enter element 3: 6
Enter element 4: 8
Enter element 5: 10
Enter element 6: 2
Enter element 7: 4
Enter element 8: 8

----- TEST REPORT -----
Elements : 8
Sum      : 44
Expected : Memory released
Result   : PASS
Cleanup  : Successful
advaith@DESKTOP-0E1OBMR:~$ |
```

OUTPUT:

21. To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance.

To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation.

A screenshot of a terminal window with a dark background. The window title bar shows 'advaith@DESKTOP-0E1OBMR: X' and a '+' icon. The terminal content shows the GNU nano 8.7.1 editor with a C program. The program defines two pipes, pipe1 and pipe2, each with two endpoints. It forks three processes. Process 1 (ls) forks p1, duplicates pipe1[1] to stdout, and runs 'ls'. Process 2 (grep) forks p2, duplicates pipe1[0] to stdin and pipe2[1] to stdout, and runs 'grep *.c'. Process 3 (wc) forks p3, duplicates pipe2[0] to stdin, and runs 'wc'. Each process has error handling with perror and exit(1).

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    int pipe1[2], pipe2[2];
    pid_t p1, p2, p3;
    int status;

    pipe(pipe1);
    pipe(pipe2);

    /* Process 1: ls */
    p1 = fork();

    if (p1 == 0) {
        dup2(pipe1[1], STDOUT_FILENO);
        close(pipe1[0]);
        close(pipe1[1]);

        execlp("ls", "ls", NULL);
        perror("ls failed");
        exit(1);
    }

    /* Process 2: grep */
    p2 = fork();

    if (p2 == 0) {
        dup2(pipe1[0], STDIN_FILENO);
        dup2(pipe2[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("grep", "grep", ".c", NULL);
        perror("grep failed");
        exit(1);
    }

    /* Process 3: wc */
    p3 = fork();
```

```

p3 = fork();

if (p3 == 0) {
    dup2(pipe2[0], STDIN_FILENO);

    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    execlp("wc", "wc", "-l", NULL);
    perror("wc failed");
    exit(1);
}

/* Parent closes pipes */
close(pipe1[0]);
close(pipe1[1]);
close(pipe2[0]);
close(pipe2[1]);

/* Monitor processes */
waitpid(p1, &status, 0);
printf("Job 1 completed.\n");

waitpid(p2, &status, 0);
printf("Job 2 completed.\n");

waitpid(p3, &status, 0);
printf("Job 3 completed.\n");

printf("\nPipeline execution completed successfully.\n");

return 0;

```

CODE: }

```

gcc mp.c -o program
advaith@DESKTOP-0E10BMR:~$ nano mp.c
advaith@DESKTOP-0E10BMR:~$ gcc mp.c -o program
advaith@DESKTOP-0E10BMR:~$ ./program
Job 1 completed.
36
Job 2 completed.
Job 3 completed.

Pipeline execution completed successfully.
advaith@DESKTOP-0E10BMR:~$ |

```

OUTPUT: